

## Explicacion ejemplos GSO

1) Funcion esfera  $f(x) = \sum_{i=1}^n x_i^2$

Esta funcion tiene su optimo global en  $x = (0,0,...,0)$  con  $f(0) = 0$   
clc; clear; close all;

% 1) Parámetros y función objetivo

```
N = 40;      % tamaño de la población
G = 4;       % número de grupos
D = 2;       % dimensión del espacio de búsqueda
MaxIter = 100; % iteraciones máximas
Lb = -10; Ub = 10; % límites
f = @(x) sum(x.^2); % función esfera
```

### Bloque 1 — Parámetros y función objetivo

2. **N = 40**; — define el número total de individuos en la población.
3. **G = 4**; — número de grupos en los que dividirás la población.
4. **D = 2**; — dimensión del problema (por ejemplo 2D).
5. **MaxIter = 100**; — número máximo de generaciones/iteraciones.
6. **Lb = -10; Ub = 10**; — límite inferior y superior para cada componente del vector solución.
7. **f = @(x) sum(x.^2)**; — función objetivo (aquí la esfera). @ crea una función anónima; **sum(x.^2)** suma los cuadrados de las componentes.

% 2) Inicialización población y grupos

```
P = Lb + rand(N, D).*(Ub-Lb); % P = {x1,...,xN}
fitness = arrayfun(@(i) f(P(i,:)), 1:N)';
groupSize = floor(N/G);
groups = cell(G,1);
for k = 1:G
    idx = (k-1)*groupSize + (1:groupSize);
    if k==G, idx = (k-1)*groupSize + (1:(N-(groupSize*(G-1)))); end
    groups{k} = idx;
end
```

### Bloque 2 — Inicialización de población y grupos

8. **P = Lb + rand(N, D).\*(Ub-Lb)**;
  - Crea la población **P** con **N** vectores aleatorios en **D** dimensiones, uniformes en [**Lb**, **Ub**]. **P** es una matriz **N x D**.
9. **fitness = arrayfun(@(i) f(P(i,:)), 1:N)'**;
  - Calcula el fitness (valor objetivo) de cada individuo y lo guarda en un vector columna **fitness**. **fitness(i)** es **f(P(i,:))**.
10. **groupSize = floor(N/G)**;
  - Tamaño base de cada grupo.

11. `groups = cell(G,1);`

- `groups` es un arreglo de celdas que contiene los índices (en `P`) que pertenecen a cada grupo.

12. `for k = 1:G ... end` (el bloque que llena `groups`)

- Reparte los índices `1..N` en `G` grupos lo más iguales posible; el último grupo toma el sobrante si `N` no está divisible por `G`.

% 3) Bucle principal (t = 1..MaxIter)

`bestHistory = zeros(MaxIter,1);`

`globalBest = inf; globalBestPos = NaN(1,D);`

`for t = 1:MaxIter`

`prevGlobal = globalBest; % para calcular mejora`

`for k = 1:G`

`idxs = groups{k}; % miembros del grupo k`

`% L_k = mejor(x in g_k)`

`[minFit, minIdxRel] = min(fitness(idxs));`

`leaderIdx = idxs(minIdxRel); % índice en P de L_k`

`Lk = P(leaderIdx, :); % posición del líder`

`% Para cada individuo x_i in g_k`

`for j = 1:numel(idxs)`

`i = idxs(j); % índice real en P`

`if i == leaderIdx`

`eps = 1e-3; % pequeño movimiento local`

`candidate = Lk + eps*randn(1,D);`

`else`

`alpha = 0.6; beta = 0.2;`

`candidate = P(i,:) + alpha*(Lk - P(i,:)) + beta*randn(1,D);`

`end`

`% acotar frontera`

`candidate = max(min(candidate, Ub), Lb);`

`% si mejora, actualiza individuo`

`cFit = f(candidate);`

`if cFit < fitness(i)`

`P(i,:) = candidate;`

`fitness(i) = cFit;`

`end`

`end`

`end`

`% actualizar L* (mejor líder global / mejor solución del grupo)`

`[currentBestFit, idxBest] = min(fitness);`

`if currentBestFit < globalBest`

`globalBest = currentBestFit;`

`globalBestPos = P(idxBest,:);`

`end`

```

% registro e impresión por generación
improvementPct = 0;
if isfinite(prevGlobal) && prevGlobal < inf
    improvementPct = (prevGlobal - globalBest)/abs(prevGlobal) * 100;
end
bestHistory(t) = globalBest;
fprintf('Gen %3d | Mejor fitness: %.6e | Mejora: %.4f%%\n', t, globalBest,
improvementPct);
end

```

### Bloque 3 — Bucle principal de iteraciones

13. `bestHistory = zeros(MaxIter,1);`

- Vector para almacenar el mejor fitness por generación (útil para graficar).

14. `globalBest = inf; globalBestPos = NaN(1,D);`

- Inicializa el mejor global como infinito (a minimizar) y la posición asociada vacía.

15. `for t = 1:MaxIter`

- Empieza el lazo de generaciones (tu *Mientras* `t < t_max`).

16. `prevGlobal = globalBest;`

- Guarda el fitness global anterior para poder calcular el porcentaje de mejora en esta generación.

17. `for k = 1:G`

- Itera sobre cada grupo `g_k`.

18. `idxs = groups{k};`

- `idxs` contiene los índices (filas de `P`) que pertenecen al grupo `k`.

19. `[minFit, minIdxRel] = min(fitness(idxs));`

- Encuentra el *mejor fitness dentro del grupo* y la posición relativa dentro del vector `idxs`.

20. `leaderIdx = idxs(minIdxRel);`

- Convierte la posición relativa en índice global (el índice del líder en la matriz `P`).

21. `Lk = P(leaderIdx, :);`

- Obtiene las coordenadas del líder del grupo (posición `L_k`).

22. `for j = 1:numel(idxs)`

- Recorre cada miembro del grupo (esto corresponde a tu *Para cada individuo*  $x_i \in g$  ).

23. `i = idxs(j);`

- Índice real del individuo en `P`.

24. `if i == leaderIdx`

- Si el individuo es el líder:

25. `eps = 1e-3; candidate = Lk + eps*randn(1,D);`

- El líder hace un movimiento local muy pequeño (búsqueda fina). `randn` aporta un ruido gaussiano pequeño.

26. `else`

- Si no es líder:
- 27. `alpha = 0.6; beta = 0.2; candidate = P(i,:) + alpha*(Lk - P(i,:)) + beta*randn(1,D);`
  - Movimiento hacia el líder con factor `alpha` y algo de aleatoriedad `beta*randn`. Esto corresponde a  $x_i' = x_i + \alpha \cdot (L_k - x_i) + \beta \cdot \text{rand}()$  de tu pseudocódigo (aquí usamos ruido gaussiano).
- 28. `candidate = max(min(candidate, Ub), Lb);`
  - Acota la candidata a los límites permitidos (clipping).
- 29. `cFit = f(candidate);`
  - Evalúa el fitness de la posición candidata.
- 30. `if cFit < fitness(i)`
  - Si la nueva posición es mejor (recordando que minimizamos):
- 31. `P(i,:) = candidate; fitness(i) = cFit;`
  - Actualiza la posición y su fitness (guardas la mejor encontrada).
- 32. (fin bucle individuos y fin bucle grupos) — se repite para todos los grupos y sus miembros.
- 33. `[currentBestFit, idxBest] = min(fitness);`
  - Encuentra el mejor fitness entre **todos** los individuos tras actualizar.
- 34. `if currentBestFit < globalBest`
- 35. `globalBest = currentBestFit; globalBestPos = P(idxBest,:);`
  - Si hay mejora global, actualiza  $L^*$  (mejor líder/solución global).
- 36. `improvementPct = 0; if isfinite(prevGlobal) && prevGlobal < inf ...` cálculo del porcentaje:
  - Calcula % de mejora respecto a la generación anterior:
 
$$\text{improvementPct} = \frac{\text{prevGlobal} - \text{globalBest}}{|\text{prevGlobal}|} \times 100$$
    - Si `prevGlobal` era `inf` (en la primera iteración) ponemos 0.
- 17. `bestHistory(t) = globalBest;`
  - Guarda el mejor fitness de esta generación para graficar.
- 18. `fprintf('Gen %3d | Mejor fitness: %.6e | Mejora: %.4f%%\n', t, globalBest, improvementPct);`
  - Imprime en terminal: número de generación, mejor fitness actual y porcentaje de mejora respecto a la generación anterior. Esto responde a tu petición de ver generaciones en consola.

% 4) Resultado final

```
fprintf('\nMejor fitness final: %.12e\n', globalBest);
```

```
fprintf('Mejor vector encontrado: %s\n', mat2str(globalBestPos,6));
```

#### Bloque 4 — Resultado final

```
39. fprintf('\nMejor fitness final: %.12e\n', globalBest);
```

- Imprime el fitness final (con más precisión).

```
40. fprintf('Mejor vector encontrado: %s\n',
mat2str(globalBestPos,6));
- Imprime el vector solución encontrado (posiciones).
```

**2)** función de rosenbrock:  $f(x,y) = (1 - x)^2 + 100(y - x^2)^2$

Su minimo global esta en  $(x,y) = (1,1)$

con valor:  $f(1,1) = 0$

```
function gso_rosenbrock_animado()
% Parámetros
num_individuals = 20;
max_iter = 60;
dim = 2;
lb = -3;
ub = 3;
% Función de Rosenbrock
f = @(x,y) (1-x).^2 + 100*(y - x.^2).^2;
% Inicializar población aleatoria
population = lb + (ub-lb) * rand(num_individuals, dim);
% Evaluar población
fitness = arrayfun(@(i) f(population(i,1),population(i,2)),1:num_individuals);
% Preparar figura
figure; hold on;
[X,Y] = meshgrid(lb:0.05:ub, lb:0.05:ub);
Z = f(X,Y);
contour(X,Y,Z,50); % mapa de contornos
colormap(parula);
scatter_points = scatter(population(:,1), population(:,2), 60, 'filled', 'r');
producer_mark = plot(population(1,1), population(1,2), 'kp', 'MarkerSize', 14,
'MarkerFaceColor','y');
title('GSO optimizando Rosenbrock (2D animado)');
xlabel('x'); ylabel('y');
drawnow;
for iter = 1:max_iter

    % Encontrar Producer
    [~, best_idx] = min(fitness);
    producer = population(best_idx, :);

    % Actualizar Producer en la gráfica
    set(producer_mark, 'XData', producer(1), 'YData', producer(2));
    % SCROUNGERS
    for i = 1:num_individuals
        if i ~= best_idx
            population(i,:) = population(i,:) + rand * (producer - population(i,:));
        end
    end
end
```

```

% RANGERS (exploración aleatoria)
num_rangers = round(0.2*num_individuals);
rangers_idx = randperm(num_individuals, num_rangers);
population(rangers_idx,:) = lb + (ub-lb)*rand(num_rangers, dim);
% Reevaluar fitness
fitness = arrayfun(@(i) f(population(i,1),population(i,2)),1:num_individuals);

% Actualizar puntos en la animación
set(scatter_points, 'XData', population(:,1), 'YData', population(:,2));

% Mostrar progreso
fprintf("Iteración %d | Mejor fitness = %.6f\n", iter, min(fitness));

pause(0.05);
end
% Resultado final
[best_val, idx_best] = min(fitness);
best_sol = population(idx_best,:);
fprintf("\nMejor solución encontrada: (%.6f, %.6f)\n", best_sol(1), best_sol(2));
fprintf("Valor de la función = %.8f\n", best_val);
end
Inicialización
population = lb + (ub-lb) * rand(num_individuals, dim);
Se generan 20 individuos, cada uno con 2 valores (x, y) dentro del rango [-3,3].
Fitness.
fitness = arrayfun(@(i) f(population(i,1),population(i,2)),1:num_individuals);
Se calcula la función de Rosenbrock para cada individuo.
Producir
[~, best_idx] = min(fitness);
producer = population(best_idx, :);
Encuentra el mejor individuo o sea el que está más cerca de la solución.
Scroungers
for i = 1:num_individuals
    if i ~= best_idx
        population(i,:) = population(i,:) + rand * (producer - population(i,:));
    end
end
Cada individuo (excepto el líder):


- Se mueve un poco hacia el líder
- La distancia del movimiento depende de rand (aleatorio entre 0 y 1)

```

Esto es **explotación**: el grupo converge hacia el mejor.

## Rangers

```
num_rangers = round(0.2*num_individuals);
```

```
rangers_idx = randperm(num_individuals, num_rangers);
```

```
population(rangers_idx,:) = lb + (ub-lb)*rand(num_rangers, dim);
```

Se elige **20% aleatorio** de la población (**randperm**)

Los rangers se **recolocan en lugares nuevos aleatorios**

```
fitness = arrayfun(@(i) f(population(i,1),population(i,2)),1:num_individuals);
```

```
set(scatter_points, 'XData', population(:,1), 'YData', population(:,2));
```

Se recalcula todo y se muestra el movimiento.

Animacion

### 1) Se evalúa la función para cada individuo

Cada punto rojo tiene coordenadas (x,y).

Se evalúa: f(x,y), Un valor más bajo = mejor solución.

### 2) Se identifica al **Producer**

Entre todos los puntos, se busca: el que tenga el menor valor de f(x,y)

Ese se convierte en el líder (Producer), en la animación lo marcamos con:

★ Un marcador amarillo

### 3) Los Scroungers se mueven hacia el líder

Los demás puntos **se acercan suavemente** al líder:

$$x_i = x_i + r(Lider - x_i)$$

Ese movimiento hace que la población se vaya agrupando alrededor del mejor.

### 4) Los Rangers exploran

Un **20%** de los individuos es reemplazado por valores **aleatorios** dentro del espacio:

x = valor aleatorio dentro de [-3,3]

Esto sirve para que el algoritmo **no se quede estancado**.

## 5) La animación se actualiza

- La **estrella amarilla** se mueve si el líder cambia.
- Los **puntos rojos** se reubican.
- Se imprime el **mejor fitness** en consola.